Logic Tensor Networks for Logically Consistent Large Language Models

Anishwar Chakraborty
Travis Tran
CS 273P – Machine Learning and Data Mining
University of California, Irvine

March 2026

Abstract

Large language models (LLMs) can assign plausible truth values to natural-language statements, but they often contradict themselves when reasoning across related facts. We explore whether Logic Tensor Networks (LTNs) can reduce these inconsistencies while keeping factual accuracy intact. Our approach treats a pretrained LLM as a soft truth-value estimator and feeds its output probabilities into differentiable logical constraints covering universal quantification, conjunction, disjunction, equivalence, and exclusive-or.

The training objective combines a supervised factual loss with a logic-based constraint-satisfaction loss. We compare four configurations: (i) a *base* LLM with no logic training, (ii) an *SDD* model that uses Sentential Decision Diagrams for propositional semantic loss, (iii) an *LTN* model driven by fuzzy first-order logic constraints, and (iv) a *hybrid* model that combines both SDD and LTN losses. On BeliefBank with LLaMA 3.2-1B, SDD achieves the best factuality (F1 = 0.88) and consistency (0.87). The hybrid model reaches the highest fuzzy rule satisfaction (0.70). LTN-only fails to converge (F1 = 0.14), mainly because of sensitivity to the logic-weight hyperparameter. The untrained baseline scores near random (F1 = 0.18). These results show the promise of neuro-symbolic training for LLM belief coherence, but also reveal how hard it is to balance fuzzy-logic losses with factual supervision.

1 Introduction

1.1 Problem Definition and Motivation

Large language models pick up broad factual knowledge during pretraining, but they are prone to logical self-contradiction across related facts [4]. An LLM might correctly state that "a dog is a mammal" and "a mammal is warm-blooded," yet deny that "a dog is warm-blooded." Each answer is plausible on its own; taken together, they violate a simple transitivity rule.

Current fixes sit at two extremes. Full-scale fine-tuning is expensive and risks catastrophic forgetting. Offloading reasoning to an external symbolic solver is brittle and breaks end-to-end differentiability. We propose a middle ground using Logic Tensor Networks (LTNs). LTNs are a neuro-symbolic framework that turns first-order logic formulas into differentiable tensor computations. They rely on fuzzy logic, a generalisation of classical Boolean logic where truth values range continuously over $[0, 1]$ instead of being strictly true or false. By using LTN constraint satisfaction as a training signal, we can steer the LLM toward logically coherent beliefs without needing a separate reasoning engine at inference time.

Research question. Can LTN-style constraint satisfaction improve logical consistency without hurting factual accuracy? And does it generalise better than the existing SDD-based approach once we introduce richer rules (XOR, conjunction, disjunction, equivalence)?

1.2 Related Work

Logically Consistent LMs (LoCo-LM). Calanzone et al. [1] introduce a neuro-symbolic loss that enforces implication and negation constraints on BeliefBank using Sentential Decision Diagrams (SDDs). An SDD is a compact data structure that represents a Boolean function as a directed acyclic graph. It supports conjunction, disjunction, negation, and model counting in time polynomial in the diagram size. Their loss enumerates every satisfying assignment of a propositional formula compiled into an SDD and penalises the LLM for placing probability mass on unsatisfying worlds. SDDs are principled, but they are strictly propositional. They cannot express quantified rules, and diagram size can blow up exponentially for constraints like XOR or multi-antecedent conjunctions.

Logic Tensor Networks. LTN [2] is a framework for differentiable first-order logic. It represents logical formulas through fuzzy semantics over real-valued tensors. Quantified rules are aggregated with the pMeanError operator, a differentiable generalisation of the universal quantifier that focuses gradient pressure on the least-satisfied groundings. The LTNtorch library [3] provides a stable "product configuration" using the product t-norm for conjunction (truth values are multiplied), probabilistic sum for disjunction, and Reichenbach implication. This combination is well-suited to gradient-based optimisation.

BeliefBank. Kassner et al. [4] build a structured knowledge base of 12,363 true/false statements about 85 animal and plant entities. It comes with a constraint graph of 4,058 implication and negation edges derived from ConceptNet. We use BeliefBank as our evaluation dataset and extend its constraint graph with richer rule types.

Semantic Loss. Xu et al. [5] derive a differentiable loss that bridges neural-network outputs with propositional constraints. It sums over the satisfying models of a formula compiled into an SDD. Both LoCo-LM and our own SDD and hybrid configurations rely on this loss.

2 Methodology

2.1 Dataset Description

BeliefBank (original). The original dataset has 12,363 true/false statements about 85 entities (animals and plants). A constraint graph links these statements with 4,058 directed edges, each encoding an implication or negation (e.g. `IsA,mammal` ⇒ `HasProperty,warmblooded`). We feed this graph directly into the SDD-based experiments.

BeliefBank Augmented. We extend the constraint set with approximately 600–700 new rules of five types not expressible as simple binary implications:

- **XOR**: exactly one of a set of predicates should hold (e.g. an entity is either domestic or wild, but not both);
- **AND-implies**: a conjunction of antecedents implies a consequent;
- **OR-implies**: a disjunction of antecedents implies a consequent;
- **Equivalence**: two predicates are logically equivalent;
- **Universal**: a single-antecedent implication.

We generate these rules by prompting an LLM with the full truth table of each entity's predicates and then validate them against the silver-fact labels. After deduplication, the combined rule set (legacy + augmented) is used for both LTN and hybrid training and for evaluation across all configurations.

Data splits. Train/validation/test splits are done by entity (not randomly by fact) so that generalisation is tested on unseen entities. In practice, antecedent predicates form the training set and consequent-only predicates form the test set, with 10% of the antecedent facts held out for validation.

2.2 Model Design

2.2.1 Base LLM as a Soft Truth Estimator

We use LLaMA 3.2-1B [8] as the backbone. To fit the model in GPU memory we load it in 4-bit NF4 quantisation via BitsAndBytes, which stores each weight in only 4 bits and cuts memory use by roughly 4×. On top of the frozen quantised weights we apply Low-Rank Adaptation (LoRA, r=8, α=2, dropout = 0.05). LoRA injects small trainable low-rank matrices into every linear layer, so only a tiny fraction of parameters are updated during fine-tuning.

Given a natural-language statement s, we build a prompt of the form

```
You can answer only with "yes" or "no".  Is the fact true?
Fact:  {s} Answer:
```

We then extract the softmax probability of the target token ("yes" or "true", depending on prompt format) at the last position:

$$P_\theta(s) \;=\; \mathrm{softmax}(\mathbf{h}_L^{(\mathrm{last})}\, W_{\mathrm{vocab}})[\texttt{target_token}], \tag{1}$$

Here $\mathbf{h}_L^{(\mathrm{last})}$ is the hidden state at the final position of the L-layer transformer and W_{vocab} is the unembedding matrix. Because the whole pipeline is differentiable with respect to the LoRA parameters, $P_\theta(s)$ can serve as the grounding for all downstream logical operators.

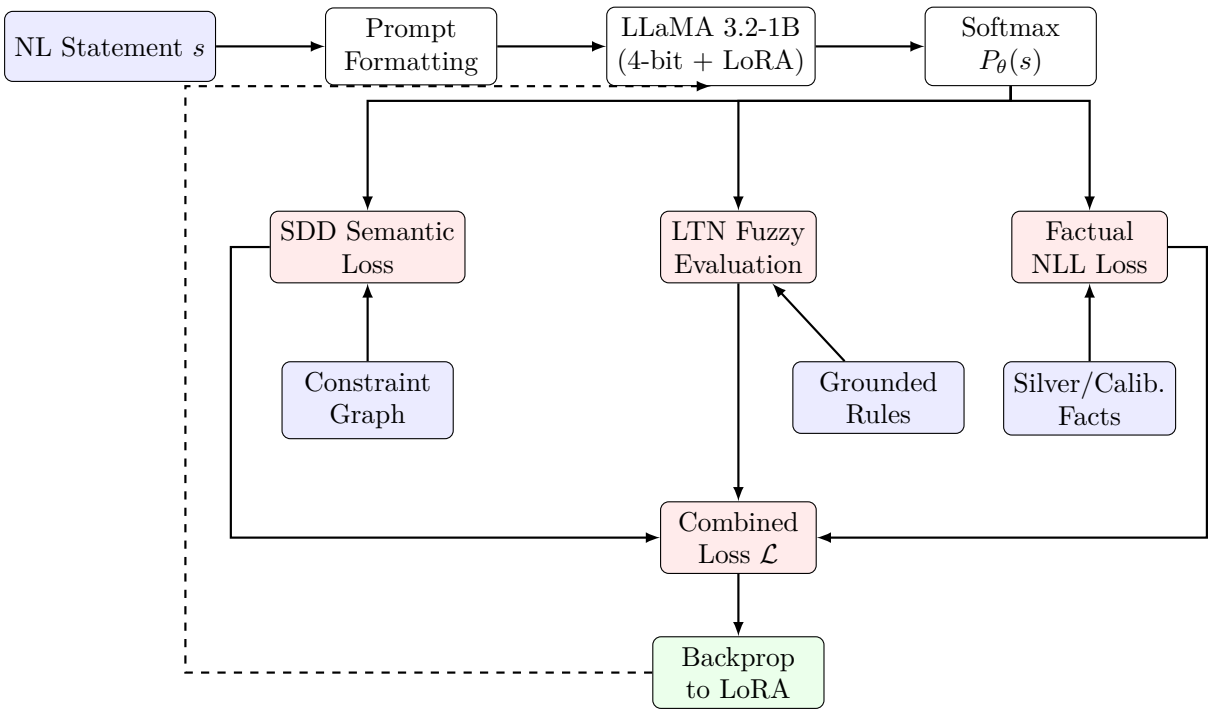


Figure 1: System architecture. A statement is formatted into a prompt and passed through the quantised LLaMA model. The output probability $P_\theta(s)$ feeds into three loss components, whose combined gradient updates only the LoRA adapters.

2.2.2 SDD Semantic Loss

Following Xu et al. [5] and Calanzone et al. [1], we compile each constraint into an SDD and enumerate its satisfying models. For a formula φ over Boolean variables $x_1, \ldots, x_n$ the semantic loss is

$$\mathcal{L}_{\text{SDD}} \;=\; -\log\left(\sum_{w \models \varphi} \prod_{i=1}^{n} p_i^{w_i}\,(1-p_i)^{1-w_i} + \epsilon\right), \tag{2}$$

where $p_i = P_\theta(s_i)$ is the LLM's probability for statement s_i, $w_i \in \{0,1\}$ is the truth assignment in world w, and $\epsilon = 10^{-10}$ prevents numerical underflow.

Our SDD constraint manager supports four modes, combined in the `all` configuration used in experiments:

$$\text{Implication:} \quad \neg x_1 \vee x_2, \tag{3}$$

$$\text{Inverse impl.:} \quad x_2 \vee \neg x_1, \tag{4}$$

$$\text{Negation:} \quad (x_i \oplus \overline{x}_i) \text{ for both } i \in \{1,2\}, \tag{5}$$

$$\text{All:} \quad (3) \wedge (4) \wedge (5). \tag{6}$$

When ground-truth labels are available for either side of a constraint, we condition the SDD on those assignments. This restricts the set of satisfying worlds and gives a stronger training signal.

2.2.3 LTN Fuzzy Logic Evaluation

We use the stable product configuration from LTNtorch [3]. Every truth value is clamped to $[\epsilon, 1-\epsilon]$ before each operation so that gradients do not vanish.

Fuzzy connectives.

$$\neg_f(x) = 1 - x, \tag{7}$$

$$\wedge_f(x_1, \ldots, x_k) = \prod_{i=1}^{k} \text{clamp}(x_i), \tag{8}$$

$$\vee_f(x_1, \ldots, x_k) = 1 - \prod_{i=1}^{k} (1 - \text{clamp}(x_i)), \tag{9}$$

$$(A \Rightarrow_f C) = A \cdot C + (1 - A) \cdot \gamma, \qquad \gamma = 0.5, \tag{10}$$

$$(L \Leftrightarrow_f R) = 1 - |L - R|, \tag{11}$$

where $\text{clamp}(x) = \min(\max(x, \epsilon), 1-\epsilon)$. The implication floor γ=0.5 replaces the vacuous-truth behaviour of standard Reichenbach implication ($1-A+AC$). Without it, rules whose antecedent is near zero would be trivially satisfied and produce no useful gradient. The floor keeps the truth value non-trivial and maintains gradient pressure during training.

Rule evaluation. Each rule type maps to a specific combination of connectives:

Rule type	**Fuzzy semantics**
`universal` / `and_implies`	$\wedge_f(\text{ants}) \Rightarrow_f \wedge_f(\text{cons})$
`or_implies`	$\vee_f(\text{ants}) \Rightarrow_f \wedge_f(\text{cons})$
`equivalence`	$\wedge_f(\text{ants}) \Leftrightarrow_f \wedge_f(\text{cons})$
`xor`	$1 - \text{ReLU}(\sum_i x_i - 1)$

pMeanError aggregation. Given rule truth values $u_1, \ldots, u_n$, the *pMeanError* aggregator [2] computes

$$\mathrm{pME}_p(u_1, \ldots, u_n) \;=\; 1 - \left(\frac{1}{n} \sum_{i=1}^{n} (1 - u_i)^p \right)^{1/p}, \tag{12}$$

with p=2 (the default for balanced gradient distribution). Increasing p shifts focus toward the hardest-to-satisfy rules; $p \to \infty$ recovers the minimum.

LTN logic loss.

$$\mathcal{L}_{\mathrm{LTN}} \;=\; -\log\big(\mathrm{pME}_p(u_1, \ldots, u_n)\big). \tag{13}$$

2.2.4 Factual Supervision Loss

For a set of ground-truth facts $\{(s_j, y_j)\}$ where $y_j \in \{0, 1\}$, the factual loss is the negative log-likelihood of the target token:

$$\mathcal{L}_{\mathrm{fact}} \;=\; -\frac{1}{|B|} \sum_{j \in B} \log P_\theta\big(\mathrm{token}_{y_j} \mid \mathrm{prompt}(s_j)\big), \tag{14}$$

where token_{y_j} is "yes" when y_j=1 and "no" when y_j=0.

2.2.5 Combined Training Objectives

We train three configurations with different composite losses:

SDD-only.

$$\mathcal{L}_{\text{SDD-only}} = \lambda_{\mathrm{sdd}}\, \mathcal{L}_{\mathrm{SDD}}. \tag{15}$$

Because the SDD loss already conditions on ground-truth labels when they are available, it implicitly encodes factual grounding. No separate factual term is needed.

LTN-only.

$$\mathcal{L}_{\text{LTN-only}} = \lambda_{\mathrm{ltn}}\, \mathcal{L}_{\mathrm{LTN}} + \lambda_{\mathrm{fact}}\, \mathcal{L}_{\mathrm{fact}}. \tag{16}$$

Hybrid.

$$\mathcal{L}_{\mathrm{Hybrid}} = \lambda_{\mathrm{sdd}}\, \mathcal{L}_{\mathrm{SDD}} + \lambda_{\mathrm{ltn}}\, \mathcal{L}_{\mathrm{LTN}} + \lambda_{\mathrm{fact}}\, \mathcal{L}_{\mathrm{fact}}. \tag{17}$$

In the hybrid setup, SDD handles the original binary implication and negation constraints. LTN takes care of the augmented complex rules (`and_implies`, `or_implies`, `xor`).

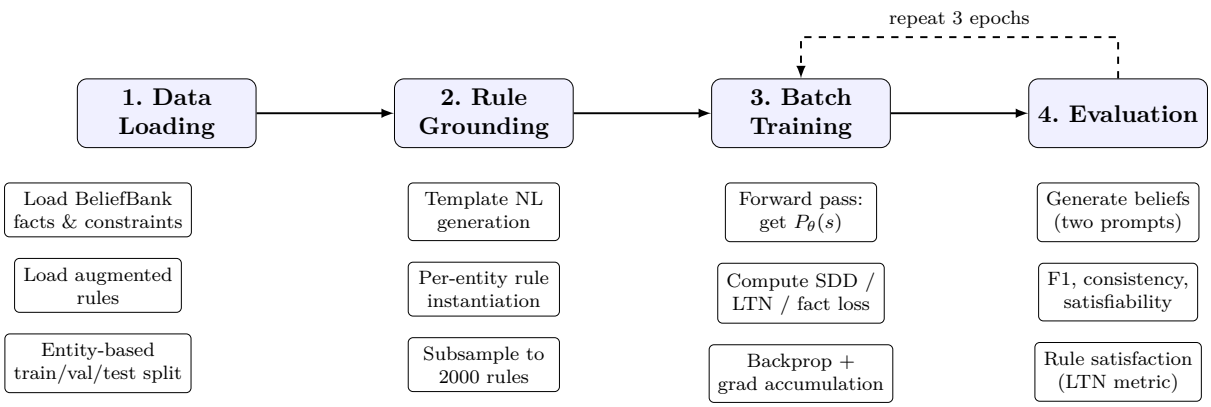


Figure 2: Training pipeline. Data is loaded and split by entity, rules are grounded into natural-language statements, batches pass through the combined loss, and evaluation runs after each epoch.

2.3 Training Procedure

All experiments fine-tune LLaMA 3.2-1B for 3 epochs on an NVIDIA A100 GPU through Google Colab. Table 1 lists the hyperparameters for each configuration.

Table 1: Hyperparameter settings for each training configuration.

Parameter	**SDD**	**LTN**	**Hybrid**
Learning rate	3×10^{-4}	1×10^{-4}	1×10^{-4}
Batch size	128	16	128
Accumulation steps	1	4	1
Effective batch	128	4	128
λ_{sdd}	1.0 (implicit)	N/A	1.0
λ_{ltn}	N/A	5.0	5.0
λ_{fact}	N/A (implicit)	1.0	1.0
LoRA rank r	8	8	8
Implication floor γ	N/A	0.5	0.5
pME power p	N/A	2.0	2.0
Max training rules	N/A	2000	2000
Prob chunk size	N/A	8	8
Gradient clipping	N/A	1.0	1.0
LR scheduler	CosineAnnealingLR		
Quantisation	4-bit NF4 (BitsAndBytes)		
Epochs	3		

SDD training loop. Each batch of grounded constraints is processed by: (1) randomly selecting one of two prompt formats, (2) computing P_θ for antecedent, consequent, and their negations, (3) compiling the SDD formula per sample, (4) computing the semantic loss (eq. (2)), and (5) backpropagating with gradient accumulation.

LTN training loop. For each batch of grounded rules: (1) collect all unique statements referenced in the batch, (2) compute P_θ for each statement in chunks of 8 (to fit GPU memory), (3) evaluate each rule's fuzzy truth value, (4) aggregate via pMeanError and compute $\mathcal{L}_{\text{LTN}}$, (5) separately compute $\mathcal{L}_{\text{fact}}$ on a mini-batch of facts, and (6) backpropagate the weighted sum with gradient clipping at $\|\nabla\|_{\max}$=1.0.

Hybrid training loop. This loop merges the two above. SDD loss covers binary constraints, LTN loss covers complex rules (`and_implies`, `or_implies`, `xor`), and a separate factual loss is computed on a fact mini-batch. The three terms are summed and backpropagated jointly.

2.4 Evaluation Metrics

We evaluate along two axes: factual accuracy and logical consistency.

Factual accuracy.

- **F1 score**: the harmonic mean of precision and recall of the model's binary belief assignments against the ground-truth labels.
- **Antecedent / Consequent F1**: F1 computed separately on antecedent and consequent fact subsets, measuring whether the model preserves accuracy on both sides of constraints.

Logical consistency (SDD-based). Given a constraint $A \Rightarrow B$:

- **Consistency**: when the reference believes A, does the model's belief about B match the constraint's expected value?
- **Inverse consistency**: when the model believes $\neg B$, does it also believe $\neg A$? (Contrapositive check.)
- **Multi-hop consistency**: for 2-hop chains $A \Rightarrow B \Rightarrow C$, when both A and B are believed, does the model's belief about C match?
- **Satisfiability**: does the model's joint assignment (A, B) lie in the truth table of the implication formula?
- **Negation consistency**: does $P(s) \neq P(\neg s)$ for every statement s and its negation?

Rule satisfaction (LTN-based). Average rule satisfaction is the mean fuzzy truth value across all applicable grounded rules, using the model's binary beliefs as crisp inputs. It tells us how well the model respects the full augmented rule set, including complex rules that the SDD metrics cannot capture.

3 Experimental Results

3.1 Experimental Setup

All four configurations share the same LLaMA 3.2-1B backbone, the same BeliefBank data splits, and the same entity-based subsampling (30 entities for evaluation). We evaluate with two prompt formats on two fact sets (calibration and silver). The main tables report the average across both prompts; per-prompt breakdowns are included for completeness.

3.2 Performance of Different Approaches

Tables 2 and 3 present the full results on calibration and silver facts respectively.

Table 2: Test results on **calibration** facts (average of prompt 0 and prompt 1). Best value in each column is **bolded**. "N/A" indicates the metric is not available for that configuration.

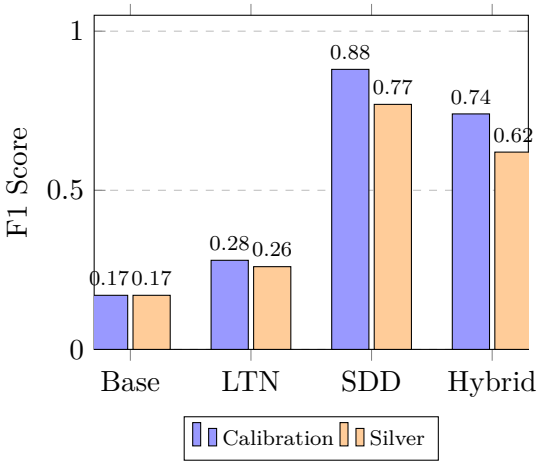
Metric	Base	LTN	SDD	Hybrid
F1	0.17	0.28	**0.88**	0.74
Negation consistency	0.21	0.32	**0.95**	0.85
Avg. rule satisfaction	0.65	0.62	0.59	**0.70**
Self-consistency	N/A	N/A	**0.87**	0.77
Self-inverse consistency	N/A	N/A	**0.94**	0.90
Self-multihop consistency	N/A	N/A	**0.94**	0.84
Consistency (ref)	N/A	N/A	**0.87**	0.77
Inverse consistency (ref)	N/A	N/A	**0.94**	0.90
Multihop consistency (ref)	N/A	N/A	**0.94**	0.84
Satisfiability	N/A	N/A	**0.98**	0.96
Antecedents F1	0.09	0.07	**0.91**	0.78
Consequents F1	0.21	0.44	**0.84**	0.74

Table 3: Test results on **silver** facts (average of prompt 0 and prompt 1). Best value in each column is **bolded**.

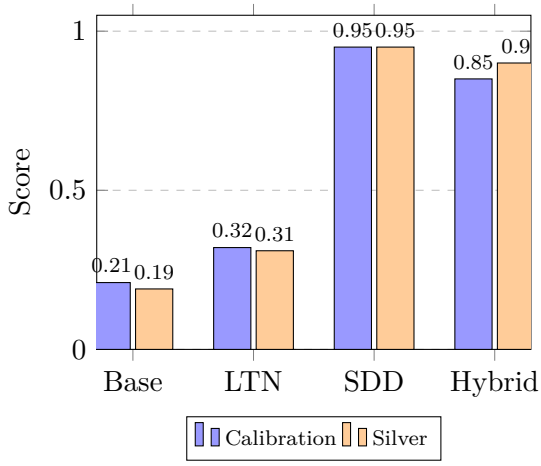
Metric	Base	LTN	SDD	Hybrid
F1	0.17	0.26	**0.77**	0.62
Negation consistency	0.19	0.31	**0.95**	0.90
Avg. rule satisfaction	**0.69**	0.68	0.55	0.69
Self-consistency	N/A	N/A	**0.83**	0.73
Self-inverse consistency	N/A	N/A	**0.93**	0.89
Self-multihop consistency	N/A	N/A	**0.92**	0.79
Consistency (ref)	N/A	N/A	**0.76**	0.67
Inverse consistency (ref)	N/A	N/A	**0.93**	0.89
Multihop consistency (ref)	N/A	N/A	**0.92**	0.83
Satisfiability	N/A	N/A	**0.98**	0.96
Antecedents F1	0.07	0.08	**0.85**	0.68
Consequents F1	0.23	0.43	**0.72**	0.60

Table 4: Per-prompt breakdown of key metrics on calibration facts. Prompt 0 uses "true"/"false" labels; Prompt 1 uses "yes"/"no".

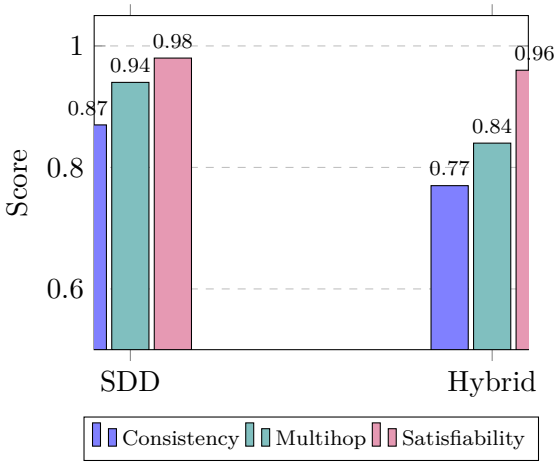
Prompt	Metric	Base	LTN	SDD	Hybrid
0	F1	0.18	0.14	0.88	0.71
	Neg. consist.	0.23	0.14	0.94	0.86
	Rule sat.	0.62	0.63	0.58	0.69
1	F1	0.16	0.43	0.89	0.77
	Neg. consist.	0.18	0.50	0.96	0.85
	Rule sat.	0.68	0.62	0.60	0.72



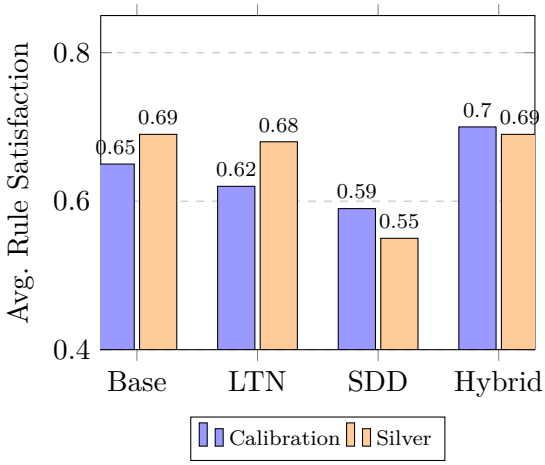
(a) Factual accuracy (F1).



(b) Negation consistency.



(c) SDD-based consistency metrics (calibration).



(d) Average rule satisfaction (LTN metric).

Figure 3: Comparison of all four model configurations across key metrics.

4 Analysis and Discussion

4.1 SDD Dominance on Factuality and Consistency

The SDD-only model leads on every factuality and consistency metric. Its semantic loss (eq. (2)) works in an exact probabilistic regime. It enumerates all satisfying worlds, so the gradient signal is precise. The loss also conditions on ground-truth labels when they are available, which effectively folds factual supervision and logical regularisation into one term. A large batch size (128) keeps gradient estimates stable, and the higher learning rate (3×10^{-4}) lets the model converge within the 3-epoch budget.

4.2 LTN Convergence Failure

The LTN-only model performs worst among the trained configurations. Its F1 actually drops below the untrained base on prompt 0 (0.14 vs. 0.18). We trace this to three interacting causes:

Logic weight sensitivity. At λ_{ltn}=5 and λ_{fact}=1 the fuzzy logic loss dominates the objective. The model learns to output near-constant probabilities (all low or all high) that satisfy many

rules vacuously. Predicting $P_\theta(s) \approx 0$ for every statement, for example, makes most implications trivially true because the antecedent is false, but it destroys factual accuracy.

Setting the logic weight lower (e.g. $\lambda_{\text{ltn}}=1$) produces the opposite problem. Not enough constraint pressure, and the model converges to a purely factual solution with no logical gain. Finding the right balance proved difficult within our hyperparameter budget.

Small effective batch size. Memory constraints forced an effective batch size of 4 (batch 16 with 4× gradient accumulation), producing high-variance gradients. The SDD and hybrid configurations use batch 128, yielding far more stable updates.

Probability chunking overhead. Computing P_θ for every unique statement in a rule batch requires processing them in chunks of 8 due to memory limits. This issues many small forward passes, which slows training. Worse, each backward pass only sees a fraction of the constraint graph, so the gradient signal lacks coherence.

4.3 Hybrid Trade-offs

The hybrid model reaches the **highest average rule satisfaction** (0.70 on calibration, 0.69 on silver). This confirms that the LTN component adds meaningful gradient signal for complex rules SDDs cannot handle. On the other hand, factuality (F1 = 0.74 on calibration) and consistency fall short of SDD-only. We attribute this to loss interference. Three loss terms ($\mathcal{L}_{\text{SDD}}$, $\mathcal{L}_{\text{LTN}}$, $\mathcal{L}_{\text{fact}}$) compete for the limited capacity of the LoRA adapters (r=8). The LTN loss sometimes pushes probabilities in directions that conflict with the SDD signal.

4.4 Design Challenges

SDD edge cases. The SDD loss compiles one formula per sample and enumerates its models. If a formula becomes unsatisfiable after conditioning on ground-truth labels, there are no satisfying worlds and the sum in eq. (2) hits zero. We fall back to $-\log(\epsilon)$, but these samples still produce uninformative constant gradients. Before we added this fallback, the result was `NaN` losses during training.

Batch construction asymmetry. SDD constraints are grounded per-entity and loaded through a standard `DataLoader`, giving uniform batches of 128. LTN rules are different: a single `universal` rule touches 2 statements, while a complex `and_implies` may touch 4 or more. This makes GPU memory usage unpredictable from batch to batch. Our workaround is `prob_chunk_size = 8`, which processes 8 statements at a time but adds sequential overhead.

Prompt sensitivity. Table 4 shows that the choice of prompt format ("true"/"false" vs. "yes"/"no") matters a lot. The LTN model's F1 jumps from 0.14 to 0.43 just by switching from prompt 0 to prompt 1. This suggests the model's token biases interact unpredictably with the logic loss and points to the need for prompt-robust training or an instruction-tuned base model.

5 Limitations and Future Work

Instruct-tuned base models. We use the raw LLaMA 3.2-1B base model, which has poor zero-shot instruction following. An instruct-tuned variant (e.g. Llama-3.2-1B-Instruct) would give a stronger starting point for factuality and lower prompt sensitivity.

Prompt diversity. We train and evaluate on only two prompt formats. Testing with a wider range of templates, including few-shot prompts, would reveal how robust the approach is to surface-form variation.

LTN hyperparameter tuning. The LTN-only configuration suffers from a coarse hyperparameter search. A curriculum-style schedule for λ_{ltn}, starting low and ramping up over epochs, could let the model learn factual accuracy first before logic regularisation kicks in.

Log-space fuzzy operators. The logLTN framework [7] runs fuzzy operators in logarithm space to avoid the vanishing products and underflow that plague the standard product t-norm when many conjuncts are involved. Adopting logLTN could improve LTN training stability.

Larger models and datasets. Scaling to larger LLMs (7B+) and richer knowledge bases would show whether these patterns hold more broadly. Larger LoRA ranks and longer training may also help the hybrid model overcome loss interference.

Inference-time reasoning. Our approach bakes logical constraints into the training loss. At inference time the model relies only on its learned parameters. A natural extension would pair training-time LTN regularisation with lightweight inference-time constraint propagation.

6 Conclusion

We set out to test whether Logic Tensor Networks can make LLM outputs more logically consistent. The idea is to add differentiable fuzzy-logic constraints to the fine-tuning objective. We ran experiments on BeliefBank with LLaMA 3.2-1B across four configurations: an untrained base model, an SDD model with propositional semantic loss, an LTN-only model with fuzzy first-order logic loss, and a hybrid that combines both.

SDD proved most effective overall. It reached an F1 of 0.88 and consistency of 0.87 on calibration facts, roughly 5× better than the untrained baseline. The hybrid model showed that LTN constraints add real value for complex rules (highest rule satisfaction at 0.70), but at the cost of lower factuality compared to SDD alone. LTN-only failed to converge, largely because of sensitivity to the logic-weight hyperparameter and the small batch sizes forced by memory limits.

The takeaway is that exact probabilistic constraints (SDDs) still beat approximate fuzzy methods (LTNs) when the rules fit in propositional logic. For richer first-order rules that SDDs cannot handle efficiently, the hybrid approach is a workable alternative. Future work should explore curriculum-based weight scheduling, instruct-tuned base models, and log-space fuzzy operators to narrow the gap between LTN and SDD performance.

7 Author Contributions

This project was a joint effort between Travis Tran and Anishwar Chakraborty. We began by jointly studying the LoCo-LM codebase [1] and deciding which pieces to reuse and which to build from scratch.

Travis Tran. Travis led the data-augmentation pipeline: he prompted an LLM with per-entity truth tables to generate the augmented rule set (XOR, AND-implies, OR-implies, equivalence, universal) and validated the generated rules against silver-fact labels. He also implemented and debugged the SDD training loop, including the constraint manager, the four SDD

modes (implication, inverse implication, negation, all), and the edge-case handling for unsatisfiable formulas that initially caused `NaN` losses.

Anishwar Chakraborty. Anishwar designed and implemented the LTN training pipeline: the fuzzy connectives (product t-norm, probabilistic sum, floored Reichenbach implication, equivalence), the pMeanError aggregation, the rule-grounding logic, and the probability-chunking workaround for GPU memory. He also built the hybrid training loop that merges SDD and LTN losses and ran the evaluation across all four configurations.

References

[1] Diego Calanzone, Stefano Teso, and Antonio Vergari. Logically consistent language models via neuro-symbolic integration. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[2] Samy Badreddine, Artur d'Avila Garcez, Luciano Serafini, and Michael Spranger. Logic Tensor Networks. *Artificial Intelligence*, 303:103649, 2022.

[3] Tommaso Carraro, Benedikt Wagner, Samy Badreddine, and Alessandro Daniele. LTNtorch: PyTorch implementation of Logic Tensor Networks. *arXiv preprint arXiv:2409.16045*, 2024.

[4] Nora Kassner, Oyvind Tafjord, Hinrich Schütze, and Peter Clark. BeliefBank: Adding memory to a pre-trained language model for a systematic notion of belief. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8707–8718, 2021.

[5] Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy Van den Broeck. A semantic loss function for deep learning with symbolic knowledge. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 5502–5511, 2018.

[6] Adnan Darwiche. SDD: A new canonical representation of propositional knowledge bases. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 819–826, 2011.

[7] Alessandro Daniele, Tommaso Campari, Samy Badreddine, and Luciano Serafini. logLTN: Differentiable fuzzy logic in the logarithm space. *arXiv preprint arXiv:2306.14546*, 2023.

[8] Meta AI. Llama 3.2: Lightweight text and multimodal models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Code Repository

The full source code for this project, including training scripts, configuration files, evaluation utilities, and the augmented rule set, is publicly available at:

https://github.com/travis-tran03/CS273P-LTN-LLM